Document number:	P0663R0
Date:	2017-06-19
Project:	C++ Extensions for Ranges, Library Working Group
Reply-to:	Casey Carter <Casey@Carter.net>

Ranges TS “Ready” Issues for the July 2017 (Toronto) meeting

155: Comparison concepts and reference types

Many concepts in the TS are well-defined over value types - un-cv-qualified non-array object types - but have unclear meaning for non-value types. For example, `EqualityComparable<foo>()` has a straight-forward meaning for a value type `foo`:

- `bool(a == b)` iff `a` equals `b` (== means “equals”)
- `bool(a != b) == !bool(a == b)` (As relations, `!=` is the complement of ==)
- `a == b` and `a != b` are valid non-modifying equality-preserving expressions if both `a` and `b` are expressions with type `foo` or `const foo` and any value category.

What meaning, if any, should `EqualityComparable<const foo>()` have? `EqualityComparable<foo&>()`? `EqualityComparable<volatile foo&&>()`?

P0547 corrects the problem in the definitions of the Object concepts, but not the Comparison concepts.

Proposed Resolution

Change the definition of the `Booolean` concept ([concepts.lib.compare.boolean]/p1) as follows (includes the resolution for #330):

```
template <class B>
concept bool Boolean() {
-  return MoveConstructible<B>() && // (see 4.5.4)
-    requires(const B b1, const B b2, const bool a) {
-      bool(b1);
-      { b1 } -> bool;
-      bool(!b1);
-      { !b1 } -> bool;
-      { b1 && b2 } -> Same<bool>;
-      { b1 && a } -> Same<bool>;
-      { a && b1 } -> Same<bool>;
-      { b1 || b2 } -> Same<bool>;
-      { b1 || a } -> Same<bool>;
-      { a || b1 } -> Same<bool>;
-      { b1 == b2 } -> bool;
-      { b1 != b2 } -> bool;
-      { b1 == a } -> bool;
-      { a == b1 } -> bool;
-      { b1 != a } -> bool;
-      { a != b1 } -> bool;
+  return Movable<decay_t<B>>() && // (see \ref{concepts.lib.object.movable})
+    requires(const remove_reference_t<B>& b1,
+             const remove_reference_t<B>& b2, const bool a) {
+      { b1 } -> ConvertibleTo<bool>&&;
+      { !b1 } -> ConvertibleTo<bool>&&;
```

```
+      { b1 && a } ->  Same<bool>&&;
+      { b1 || a } ->  Same<bool>&&;
+      { b1 && b2 } -> Same<bool>&&;
+      { a && b2  } -> Same<bool>&&;
+      { b1 || b2 } -> Same<bool>&&;
+      { a || b2  } -> Same<bool>&&;
+      { b1 == b2 } -> ConvertibleTo<bool>&&;
+      { b1 == a  } -> ConvertibleTo<bool>&&;
+      { a == b2  } -> ConvertibleTo<bool>&&;
+      { b1 != b2 } -> ConvertibleTo<bool>&&;
+      { b1 != a  } -> ConvertibleTo<bool>&&;
+      { a != b2  } -> ConvertibleTo<bool>&&;
    };
 }
```

Change [concepts.lib.compare.boolean]/p2 as follows (depends on the resolution of #167):

```
-2 Given values b1 and b2 of type B, then Boolean<B>() is satisfied if and only if
+2 Given const lvalues b1 and b2 of type remove_reference_t<B>, then
+  Boolean<B>() is satisfied if and only if
-(2.1) — bool(b1) == [](bool x) { return x; }(b1).
 (2.2) — bool(b1) == !bool(!b1).
 (2.3) — (b1 && b2), (b1 && bool(b2)), and (bool(b1) && b2) are all equal to
         (bool(b1) && bool(b2)), and have the same short-circuit evaluation.
 (2.4) — (b1 || b2), (b1 || bool(b2)), and (bool(b1) || b2) are all equal to
         (bool(b1) || bool(b2)), and have the same short-circuit evaluation.
 (2.5) — bool(b1 == b2), bool(b1 == bool(b2)), and bool(bool(b1) == b2) are
         all equal to (bool(b1) == bool(b2)).
 (2.6) — bool(b1 != b2), bool(b1 != bool(b2)), and bool(bool(b1) != b2) are
         all equal to (bool(b1) != bool(b2)).
```

Change concept `WeaklyEqualityComparable` ([concepts.lib.compare.equalitycomparable]) as follows (includes the resolution for #330):

```
 template <class T, class U>
 concept bool WeaklyEqualityComparable() {
-  return requires(const T& t, const U& u) {
-    { t == u } -> Boolean;
-    { u == t } -> Boolean;
-    { t != u } -> Boolean;
-    { u != t } -> Boolean;
+  return requires(const remove_reference_t<T>& t,
+                  const remove_reference_t<U>& u) {
+    { t == u } -> Boolean&&;
+    { t != u } -> Boolean&&;
+    { u == t } -> Boolean&&;
+    { u != t } -> Boolean&&;
   };
 }
```

Change [concepts.lib.compare.equalitycomparable]/p1 as follows:

```
-1 Let t and u be objects of types T and U.
+1 Let t and u be const lvalues of types remove_reference_t<T> and remove_reference_t<U>.
   WeaklyEqualityComparable<T, U>() is satisfied if and only if:
 (1.1) — t == u, u == t, t != u, and u != t have the same domain.
 (1.2) — bool(u == t) == bool(t == u).
 (1.3) — bool(t != u) == !bool(t == u).
```

Change cross-type concept `EqualityComparable` ([concepts.lib.compare.equalitycomparable]) as follows (includes the resolution for #330):

```
 template <class T, class U>
 concept bool EqualityComparable() {
-  return CommonReference<const T&, const U&>() &&
+  return
     EqualityComparable<T>() &&
     EqualityComparable<U>() &&
-    EqualityComparable<
-        remove_cv_t<remove_reference_t<common_reference_t<const T&, const U&>>>>() &&
+    CommonReference<
+      const remove_reference_t<T>&,
+      const remove_reference_t<U>&>() &&
+    EqualityComparable<
+      common_reference_t<
+        const remove_reference_t<T>&,
+        const remove_reference_t<U>&>>() &&
     WeaklyEqualityComparable<T, U>();
 }
```

Change [concepts.lib.compare.equalitycomparable]/p4 as follows:

```
-4 Let a be an object of type T, b be an object of type U, and C be
-  common_reference_t<const T&, const U&>.
+4 Let a be a const lvalue of type remove_reference_t<T>, b be a
+  const lvalue of type remove_reference_t<U>, and C be
+  common_reference_t<const remove_reference_t<T>&,
+  const remove_reference_t<U>&>.
   Then EqualityComparable<T, U>() is satisfied if and only if:
 (4.1) — bool(a == b) == bool(C(a) == C(b)).
```

Change concept `StrictTotallyOrdered` ([concepts.lib.compare.stricttotallyordered]) as follows (includes the resolution for #330):

```
 template <class T>
 concept bool StrictTotallyOrdered() {
   return EqualityComparable<T>() &&
-    requires(const T a, const T b) {
+    requires(const remove_reference_t<T>& t,
+             const remove_reference_t<U>& u) {
-      { a < b } -> Boolean;
-      { a > b } -> Boolean;
-      { a <= b } -> Boolean;
-      { a >= b } -> Boolean;
+      { a < b } -> Boolean&&;
+      { a > b } -> Boolean&&;
+      { a <= b } -> Boolean&&;
+      { a >= b } -> Boolean&&;
     };
 }
```

Change [concepts.lib.compare.stricttotallyordered]/p1 to be:

```
-1 Let a, b, and c be objects of type T.
+1 Let a, b, and c be const lvalues of type remove_reference_t<T>.
 Then StrictTotallyOrdered<T>() is satisfied if and only if
```

```
 (1.1) — Exactly one of bool(a < b), bool(b < a), or bool(a == b) is true.
 (1.2) — If bool(a < b) and bool(b < c), then bool(a < c).
 (1.3) — bool(a > b) == bool(b < a).
 (1.4) — bool(a <= b) == !bool(b < a).
 (1.5) — bool(a >= b) == !bool(a < b).
```

Change cross-type concept `StrictTotallyOrdered` ([concepts.lib.compare.stricttotallyordered]) as follows (includes the resolution for #330):

```
 template <class T, class U>
 concept bool StrictTotallyOrdered() {
-  return CommonReference<const T&, const U&>() &&
+  return
     StrictTotallyOrdered<T>() &&
     StrictTotallyOrdered<U>() &&
-    StrictTotallyOrdered<
-      remove_cv_t<remove_reference_t<common_reference_t<const T&, const U&>>>>() &&
+    CommonReference<
+      const remove_reference_t<T>&,
+      const remove_reference_t<U>&>() &&
+    StrictTotallyOrdered<
+      common_reference_t<
+        const remove_reference_t<T>&,
+        const remove_reference_t<U>&>>() &&
     EqualityComparable<T, U>() &&
-    requires(const T t, const U u) {
+    requires(const remove_reference_t<T>& t,
+             const remove_reference_t<U>& u) {
-      { t < u } -> Boolean;
-      { t > u } -> Boolean;
-      { t <= u } -> Boolean;
-      { t >= u } -> Boolean;
-      { u < t } -> Boolean;
-      { u > t } -> Boolean;
-      { u <= t } -> Boolean;
-      { u >= t } -> Boolean;
+      { t < u } -> Boolean&&;
+      { t > u } -> Boolean&&;
+      { t <= u } -> Boolean&&;
+      { t >= u } -> Boolean&&;
+      { u < t } -> Boolean&&;
+      { u > t } -> Boolean&&;
+      { u <= t } -> Boolean&&;
+      { u >= t } -> Boolean&&;
     };
 }
```

Change [concepts.lib.compare.stricttotallyordered]/p2 as follows:

```
-2 Let t be an object of type T, u be an object of type U,
+2 Let t be a const lvalue of type remove_reference_t<T>,
+  u be a const lvalue of type remove_reference_t<U>,
-  and C be common_reference_t<const T&, const U&>.
+  and C be common_reference_t<const remove_reference_t<T>&,
+  const remove_reference_t<U>&>.
   Then StrictTotallyOrdered<T, U>() is satisfied if and only if
 (2.1) — bool(t < u) == bool(C(t) < C(u)).
 (2.2) — bool(t > u) == bool(C(t) > C(u)).
 (2.3) — bool(t <= u) == bool(C(t) <= C(u)).
 (2.4) — bool(t >= u) == bool(C(t) >= C(u)).
 (2.5) — bool(u < t) == bool(C(u) < C(t)).
```

```
 (2.6) — bool(u > t) == bool(C(u) > C(t)).
 (2.7) — bool(u <= t) == bool(C(u) <= C(t)).
 (2.8) — bool(u >= t) == bool(C(u) >= C(t)).
```

Change section “Concept `Relation`” ([concepts.lib.callable.relation]) as follows:

```
template <class R, class T, class U>
concept bool Relation() {
  return Relation<R, T>() &&
    Relation<R, U>() &&
-    CommonReference<const T&, const U&>() &&
-    Relation<R,
-      common_reference_t<const T&, const U&>>() &&
+    CommonReference<
+      const remove_reference_t<T>&,
+      const remove_reference_t<U>&>() &&
+    Relation<R,
+      common_reference_t<
+        const remove_reference_t<T>&,
+        const remove_reference_t<U>&>>() &&
    Predicate<R, T, U>() &&
    Predicate<R, U, T>();
}

-1 Let r be any object of type R,
+1 Let r be an expression such that decltype((r)) is R,
-  a be any object of type T,
+  a be an expression such that decltype((a)) is T,
- b be any object of type U,
+  b be an expression such that decltype((b)) is U,
-  and C be common_reference_t<const T&, const U&>.
+  and C be common_reference_t<const remove_reference_t<T>&,
+  const remove_reference_t<U>&>.
  Then Relation<R, T, U>() is satisfied if and only if
(1.1) — bool(r(a, b)) == bool(r(C(a), C(b))).
(1.2) — bool(r(b, a)) == bool(r(C(b), C(a))).
```

Change “Concept `Swappable`” ([concepts.lib.corelang.swappable]) as follows:

```
template <class T>
concept bool Swappable() {
  return requires(T&& a, T&& b) {
    ranges::swap(std::forward<T>(a), std::forward<T>(b));
  };
}

template <class T, class U>
concept bool Swappable() {
  return Swappable<T>() &&
  Swappable<U>() &&
- CommonReference<const T&, const U&>() &&
+ CommonReference<
+   const remove_reference_t<T>&,
+   const remove_reference_t<U>&>() &&
  requires(T&& t, U&& u) {
    ranges::swap(std::forward<T>(t), std::forward<U>(u));
    ranges::swap(std::forward<U>(u), std::forward<T>(t));
  };
}
```

172: `tagged<Base...>` should be implicitly constructible from `Base`

`tagged<Base, Tags...>` should be implicitly convertible from `Base&&` when `MoveConstructible<Base>()` is satisfied, and from `Base const&` when `CopyConstructible<Base>()` is satisfied.

Proposed Resolution

Change the class synopsis of `tagged` in [taggedtup.tagged] as follows:

```
 template <class Base, TagSpecifier... Tags>
   requires sizeof...(Tags) <= tuple_size<Base>::value
 struct tagged :
   Base, TAGGET (tagged<Base, Tags...>, Ti, i)... { // see below
   using Base::Base;
   tagged() = default;
   tagged(tagged&&) = default;
   tagged(const tagged&) = default;
+  tagged(Base&&) noexcept(see below)
+    requires MoveConstructible<Base>();
+  tagged(const Base&) noexcept(see below)
+    requires CopyConstructible<Base>();
   tagged &operator=(tagged&&) = default;
   tagged &operator=(const tagged&) = default;
   [...]
```

After [taggedtup.tagged]/9, add the following:

```
tagged(Base &&that) noexcept(see below )
  requires MoveConstructible<Base>();
```

10 *Remarks:* The expression in the `noexcept` is equivalent to:

```
is_nothrow_move_constructible<Base>::value
```

11 Effects: Initializes `Base` with `std::move(that)`.

```
tagged(const Base &that) noexcept(see below )
  requires CopyConstructible<Base>();
```

12 *Remarks:* The expression in the `noexcept` is equivalent to:

```
is_nothrow_copy_constructible<Base>::value
```

13 Effects: Initializes `Base` with `that`.

203: Don't slurp entities from `std` into `std::experimental::ranges::v1`

The Ranges TS, [function.objects]/p3 says:

> Any entities declared or defined directly in namespace std in header `<functional>` that are not already defined in namespace `std::experimental::ranges` in header `<experimental/ranges/functional>` are imported with *using-declarations* (7.3.3). [*Example:*

```
namespace std { namespace experimental { namespace ranges { inline namespace v1 {
using std::reference_wrapper;
using std::ref;
// ... others
}}}}
```

—*end example*]

This breaks forward compatibility should we ever decide that something in `std::` needs to have a parallel, constrained implementation in `ranges::`.

Proposed resolution

Strike paragraph 1 in [iterator.synopsis].

In [tagged.tuple]/p1, strike the text beginning with “Any entities declared or defined in namespace std in header ” and ending at the end of the example.

Strike p3 and p4 under [function.objects].

Strike p2 under [utility].

232: Kill the Readability requirement for `i++` for `InputIterators`

The requirement for `InputIterators` to provide a post-increment operator that returns a type satisfying the `Readable` concept presents significant implementation challenges for many input iterators, and makes some impossible to implement.

Proposed resolution

Adopt P0541.

235: Trivial example breaks common_type from P0022

The following test case fails with `common_type` as specified in P0022.

```
struct Int
{
    operator int();
};

// Whoops, fails:
static_assert(std::is_same_v<common_type_t<Int, int>, int>);
```

The issue is that P0022 mandates that the two template arguments be first transformed by adding `const &` to each before trying them in the conditional expression, like `decltype(true ? declval<const T&>() : declval<const U&>())`. This is sometimes desirable — e.g., it finds `int` as the common type of `reference_wrapper<int>` and `int` — but in this case it causes the conditional expression to be ill-formed since `Int`’s implicit conversion operator is not `const`-qualified.

The fix is to first try the arguments as they are (after they are decayed) to see if the conditional expression is well-formed. If not, the conditional expression is then retried with `const` and `&` qualification to see if that works.

With this fix, `common_type` as proposed passes all of libc++'s and libstdc++'s tests with the exception of one that will be broken by the adoption of LWG#2763 and two that seem related to the lookup of private class members.

Proposed Resolution

This wording is based on P0022R2. It fixes the problem noted above and also adopts some of the wording from LWG#2763.

Change Table 57 to disallow user specialization of `common_reference`. Specialization of `basic_common_reference` is the way to influence the result of `common_reference`, and it avoids the combinatorial explosion of cv- and ref-qualifiers:

Template	Condition	Comments
`template <class… T> struct common_reference;`		The member typedef `type` shall be defined or omitted as specified below. If it is ommitted, there shall be no member `type`. Each type in the parameter pack `T` shall be complete or (possibly cv) `void`. ~~A program may specialize this trait if at least one template parameter in the specialization depends on a user-defined type and `sizeof…(T) == 2`. Remark: Such specializations are needed to properly handle proxy reference types in generic code.~~

Change the description of `common_type` ([meta.trans.other]) as follows:

```
-4. For the common_type trait applied to a parameter pack [...]
+4. Note A: For the common_type trait applied to a parameter pack [...]
 [...]
 (4.3)     — Otherwise, if sizeof...(T) is two, let T1 and T2 denote the
             two types in the pack T, and let D1 and D2 be decay_t<T1> and
             decay_t<T2> respectively. Then
 (4.3.1)       — If D1 and T1 denote the same type and D2 and T2 denote
                 the same type, then
```

```
+(4.3.1.?)              — If COND_RES(T1, T2) is well-formed, then the
+                         member typedef type denotes
+                         decay_t<COND_RES(T1, T2)>.
 (4.3.1.1)              — If COMMON_REF(T1, T2) is well-formed, then the
                          member typedef type denotes that type.
 (4.3.1.2)              — Otherwise, there shall be no member type
 [...]
 (4.4.2)        — Otherwise, there shall be no member type.
+?. Note B: A program may specialize the common_type trait for two
+   cv-unqualified non-reference types if at least one of them depends on a
+   user-defined type. [Note: Such specializations are needed when only
+   explicit conversions are desired among the template arguments.
+   — end note] Such a specialization need not have a member named
+   type, but if it does, that member shall be a typedef-name for a
+   cv-unqualified non-reference type that need not otherwise meet
+   the specification set forth in Note A, above.
```

Change the description of `common_reference` ([meta.trans.other]) as follows:

```
 5. For the common_reference trait applied to a parameter pack [...]
 [...]
 (5.4.2)        — Otherwise, there shall be no member type.
+?. A program may specialize the basic_common_reference trait for
+   two cv-unqualified non-reference types if at least one of them depends on a
+   user-defined type. Such a specialization need not have a member named
+   type.
```

245: The iterator adaptors should customize `iter_move` and `iter_swap`

It may be the case that `iter_swap` of the base iterators is more efficient than `iter_move` construction + `iter_move` assignment + `move` assignment.

Proposed resolution:

In [reverse.iterator], add declarations to the synopsis:

```
      reference operator[](difference_type n) const
        requires RandomAccessIterator<I>();
    +
    + friend constexpr rvalue_reference_t<I> iter_move(const reverse_iterator& i)
    +   noexcept(see below);
    + template <IndirectlySwappable<I> I2>
    +   friend void iter_swap(const reverse_iterator& x, const reverse_iterator<I2>& y)
    +     noexcept(see below);
    +
    private:
      I current; // exposition only
    };
```

Add new sections before [reverse.iter.make] as follows:

```
    + 24.7.1.3.? iter_move [reverse.iter.iter_move]
    +
    + friend constexpr rvalue_reference_t<I> iter_move(const reverse_iterator& i)
    +   noexcept(see below);
```

```
+
+ 1 Effects: Equivalent to: return ranges::iter_move(prev(i.current));
+
+ 2 Remarks: The expression in the noexcept clause is equivalent to
+   noexcept(ranges::iter_move(declval<I&>())) && noexcept(--declval<I&>()) &&
+   is_nothrow_copy_constructible<I>::value.
+
+ 24.7.1.3.? iter_swap [reverse.iter.iter_swap]
+
+ template <IndirectlySwappable<I> I2>
+   friend void iter_swap(const reverse_iterator& x, const reverse_iterator<I2>& y)
+     noexcept(see below);
+
+ 1 Effects: Equivalent to ranges::iter_swap(prev(x.current), prev(y.current)).
+
+ 2 Remarks: The expression in the noexcept clause is equivalent to
+   noexcept(ranges::iter_swap(declval<I>(), declval<I>())) && noexcept(--declval<I
```

In [move.iterator], add declarations to the synopsis:

```
    reference operator[](difference_type n) const
      requires RandomAccessIterator<I>();

+ friend constexpr rvalue_reference_t<I> iter_move(const move_iterator& i)
+   noexcept(see below);
+ template <IndirectlySwappable<I> I2>
+   friend void iter_swap(const move_iterator& x, const move_iterator<I2>& y)
+     noexcept(see below);
+
private:
  I current; // exposition only
};
```

Add new paragraphs to [move.iter.nonmember] as follows:

```
template <RandomAccessIterator I>
  move_iterator<I>
    operator+(
      difference_type_t<I> n,
      const move_iterator<I>& x);

2 Effects: Equivalent to x + n.
+
+ friend constexpr rvalue_reference_t<I> iter_move(const move_iterator& i)
+   noexcept(see below):
+
+ -?- Effects: Equivalent to: return ranges::iter_move(i.current);
+
+ -?- Remarks: The expression in the noexcept clause is equivalent to
+     noexcept(ranges::iter_move(i.current)).
+
+ template <IndirectlySwappable<I> I2>
+   friend void iter_swap(const move_iterator& x, const move_iterator<I2>& y)
+     noexcept(see below):
+
+ -?- Effects: Equivalent to ranges::iter_swap(x.current, y.current).
+
+ -?- Remarks: The expression in the noexcept clause is equivalent to
+     noexcept(ranges::iter_swap(x.current, y.current)).
```

```
template <InputIterator I>
  move_iterator<I> make_move_iterator(I i);

3 Returns: move_iterator<I>(i).
```

In [common.iterator], add declarations to the synopsis:

```
  common_iterator& operator++();
  common_iterator operator++(int);
+
+ friend constexpr rvalue_reference_t<I> iter_move(const common_iterator& i)
+   noexcept(see below)
+     requires InputIterator<I>();
+ template <IndirectlySwappable<I> I2, class S2>
+   friend void iter_swap(const common_iterator& x, const common_iterator<I2, S2>&
+     noexcept(see below)

private:
  bool is_sentinel; // exposition only
```

Add new sections after [common.iter.op.comp]:

```
+ 24.7.4.2.? iter_move [common.iter.op.iter_move]
+
+ friend constexpr rvalue_reference_t<I> iter_move(const common_iterator& i)
+   noexcept(see below)
+     requires InputIterator<I>();
+
+ 1 Requires: !i.is_sentinel.
+
+ 2 Effects: Equivalent to: return ranges::iter_move(i.iter);
+
+ 3 Remarks: The expression in the noexcept clause is equivalent to
+   noexcept(ranges::iter_move(i.iter)).
+
+ 24.7.4.2.? iter_swap [common.iter.op.iter_swap]
+
+ template <IndirectlySwappable<I> I2>
+   friend void iter_swap(const common_iterator& x, const common_iterator<I2>& y)
+     noexcept(see below);
+
+ 1 Requires: !x.is_sentinel && !y.is_sentinel.
+
+ 2 Effects: Equivalent to ranges::iter_swap(x.iter, y.iter).
+
+ 3 Remarks: The expression in the noexcept clause is equivalent to
+   noexcept(ranges::iter_swap(x.iter, y.iter)).
```

In [counted.iterator], add declarations to the synopsis:

```
  see below operator[](difference_type n) const
    requires RandomAccessIterator<I>();

+ friend constexpr rvalue_reference_t<I> iter_move(const counted_iterator& i)
+   noexcept(see below)
+     requires InputIterator<I>();
+ template <IndirectlySwappable<I> I2>
+   friend void iter_swap(const counted_iterator& x, const counted_iterator<I2>& y)
```

```
+     noexcept(see below);
+
private:
  I current; // exposition only
```

Add new paragraphs to [counted.iter.nonmember] as follows:

```
template <RandomAccessIterator I>
  counted_iterator<I>
    operator+(difference_type_t<I> n, const counted_iterator<I>& x);

5 Requires: n <= x.cnt.

6 Effects: Equivalent to x + n.
+
+ friend constexpr rvalue_reference_t<I> iter_move(const counted_iterator& i)
+   noexcept(see below)
+     requires InputIterator<I>();
+
+ -?- Effects: Equivalent to: return ranges::iter_move(i.current).
+
+ -?- Remarks: The expression in the noexcept clause is equivalent to
+      noexcept(ranges::iter_move(i.current)).
+
+ template <IndirectlySwappable<I> I2>
+   friend void iter_swap(const counted_iterator& x, const counted_iterator<I2>& y)
+     noexcept(see below);
+
+ -?- Effects: Equivalent to ranges::iter_swap(x.current, y.current).
+
+ -?- Remarks: The expression in the noexcept clause is equivalent to
+     noexcept(ranges::iter_swap(x.current, y.current)).

template <Iterator I>
  counted_iterator<I> make_counted_iterator(I i, difference_type_t<I> n);

7 Requires: n >= 0.

8 Returns: counted_iterator<I>(i, n).
```

251: algorithms incorrectly specified in terms of `swap(*a,*b)` instead `iter_swap(a,b)`, and `move(*a)` instead of `iter_move(a)`

This change is necessary for the algorithms to properly support P0022 proxy iterators.

Proposed resolution

In [alg.move], change p1 as follows:

```
 1. Effects: Moves elements in the range [first,last) into the range
    [result, result + (last - first)) starting from first and proceeding
    to last. For each non-negative integer n < (last-first), performs
-   *(result + n) = std::move(*(first + n)).
+   *(result + n) = ranges::iter_move(first + n).
```

Change p5 as follows:

```
 5. Effects: Moves elements in the range [first,last) into the range
    [result - (last-first),result) starting from last - 1 and proceeding
    to first.6 For each positive integer n <= (last - first), performs
-   *(result - n) = std::move(*(last - n)).
+   *(result - n) = ranges::iter_move(last - n).
```

Change [alg.swap]/p1 as follows:

```
 1. Effects: For the first two overloads, let last2 be first2 + (last1 - first1).
    For each non-negative integer n < min(last1 - first1, last2 - first2) performs:
-   swap(*(first1 + n), *(first2 + n)).
+   ranges::iter_swap(first1 + n, first2 + n).
```

259: `is_swappable` type traits should not be in namespace `std`

They are defined in terms of unqualified calls to `swap`, but `swap` is unconstrained in C++14. Solution: move the `swappable` traits into the `std::experimental::ranges` namespace and define them in terms of the `ranges::swap` customization point object.

Proposed wording

To Table 1 – Ranges TS library headers, add `<experimental/ranges/type_traits>`.

From the `<type_traits>` synopsis ([meta.type.synop]), remove the declarations of:

- `is_swappable`
- `is_swappable_with`
- `is_nothrow_swappable`
- `is_nothrow_swappable_with`
- `is_swappable_v`
- `is_swappable_with_v`
- `is_nothrow_swappable_v`
- `is_nothrow_swappable_with_v`

Remove subsection “Type properties” ([meta.unary.prop]).

After subsection “Other transformations” ([meta.trans.other]), add a new subsection “Header `<experimental/ranges/type_traits>` synopsis” ([meta.type.synop.rng]) with the following:

```
namespace std { namespace experimental { namespace ranges { inline namespace v1 {
// (REF), type properties:
template <class T, class U> struct is_swappable_with;
template <class T> struct is_swappable;

template <class T, class U> struct is_nothrow_swappable_with;
template <class T> struct is_nothrow_swappable;

template <class T, class U> constexpr bool is_swappable_with_v
    = is_swappable_with<T, U>::value;
template <class T> constexpr bool is_swappable_v
    = is_swappable<T>::value;
```

```
template <class T, class U> constexpr bool is_nothrow_swappable_with_v
    = is_nothrow_swappable_with<T, U>::value;
template <class T> constexpr bool is_nothrow_swappable_v
    = is_nothrow_swappable<T>::value;
}}}}
```

After subsection “Header `<experimental/ranges/type_traits>` synopsis” ([meta.type.synop.rng]), add a new subsection “Additional type properties” ([meta.unary.prop.rng]) with the following:

[*Editor’s note:* Taken from latest C++17 working draft.]

1. These templates provide access to some of the more important properties of types.
2. It is unspecified whether the library defines any full or partial specializations of any of these templates.
3. For all of the class templates X declared in this subclause, instantiating that template with a template argument that is a class template specialization may result in the implicit instantiation of the template argument if and only if the semantics of X require that the argument must be a complete type.
4. For the purpose of defining the templates in this subclause, a function call expression declval<T>() for any type T is considered to be a trivial (3.9, 12) function call that is not an odr-use (3.2) of declval in the context of the corresponding definition notwithstanding the restrictions of 20.2.7.

Table XX - Additional type property predicates

Template	Condition	Precondition
`template <class T, class U>` `struct is_swappable_with;`	The expressions `ranges::swap(declval<T>(), declval<U>())` and `ranges::swap(declval<U>(), declval<T>())` are each well-formed when treated as an unevaluated operand (Clause 5) ~~in an overload-resolution context for swappable values (17.5.3.2)~~. Access checking is performed as if in a context unrelated to `T` and `U`. Only the validity of the immediate context of the `swap` expressions is considered. [*Note:* The compilation of the expressions can result in side effects such as the instantiation of class template specializations and function template specializations, the generation of implicitly-defined functions, and so on. Such side effects are not in the “immediate context” and can result in the program being ill-formed. *–end note*]	`T` and `U` shall be complete types, *cv* `void`, or arrays of unknown bound.
`template <class T>` `struct is_swappable;`	For a referenceable type `T`, the same result as `is_swappable_with_v<T&, T&>`, otherwise `false`.	`T` shall be a complete type, *cv* `void`, or an array of unknown bound.
`template <class T, class U>` `struct is_nothrow_swappable_with;`	`is_swappable_with_v<T, U>` is `true` and each `swap` expression of the definition of	`T` and `U` shall be complete types, *cv*

Template	Condition	Precondition
	`is_swappable_with<T, U>` is known not to throw any exceptions (5.3.7).	`void`, or arrays of unknown bound.
`template <class T>` `struct is_nothrow_swappable;`	For a referenceable type `T`, the same result as `is_nothrow_swappable_with_v<T&, T&>`, otherwise `false`.	`T` shall be a complete type, *cv* `void`, or an array of unknown bound.

286: Resolve inconsistency in `indirect_result_of`

Issue #238 repairs `indirect_result_of` in a way that makes it inconsistent. I think the error is using `IndirectInvocable` to (over-)constrain the template arguments. Really, the user of `indirect_result_of` is asking: "Can I invoke a type `F` with the reference type of a bunch of iterators?`"

In contrast, `IndirectInvocable` exists to give algorithms the leeway to call algorithms with a cross-product of iterators' value and reference types, and also check that the callable thingie is `CopyConstructible` (after #189), because that's a useful clustering of requirements when looking at how the algorithms use callables.

I suggest that `indirect_result_of` be constrained with `Invocable` instead of `IndirectlyInvocable`.

Proposed Resolution

```
template <class F, class... Is>
-    requires IndirectInvocable<decay_t<F>, Is...>()
+    requires Invocable<F, reference_t<Is>...>()
struct indirect_result_of<F(Is...)> :
-   result_of<F(reference_t<Is>...)> { };
+   result_of<F(reference_t<Is>&&...)> { };
```

`transform` gets the `CopyConstructible` requirement for its function parameter from `indirect_result_of`. We need to fix that up.

In the `<experimental/ranges/algorithm>` synopsis:

```
-template <InputIterator I, Sentinel<I> S, WeaklyIncrementable O, class F, class Proj = ident
+template <InputIterator I, Sentinel<I> S, WeaklyIncrementable O,
+    CopyConstructible F, class Proj = identity>
  requires Writable<O, indirect_result_of_t<F&(projected<I, Proj>)>>()
  tagged_pair<tag::in(I), tag::out(O)>
    transform(I first, S last, O result, F op, Proj proj = Proj{});

-template <InputRange Rng, WeaklyIncrementable O, class F, class Proj = identity>
+template <InputRange Rng, WeaklyIncrementable O, CopyConstructible F, class Proj = identity>
  requires Writable<O, indirect_result_of_t<F&(
    projected<iterator_t<R>, Proj>)>>()
  tagged_pair<tag::in(safe_iterator_t<Rng>), tag::out(O)>
    transform(Rng&& rng, O result, F op, Proj proj = Proj{});

// A.2 (deprecated):
template <InputIterator I1, Sentinel<I1> S1, InputIterator I2, WeaklyIncrementable O,
```

```
-    class F, class Proj1 = identity, class Proj2 = identity>
+    CopyConstructible F, class Proj1 = identity, class Proj2 = identity>
  requires Writable<O, indirect_result_of_t<F&(projected<I1, Proj1>,
    projected<I2, Proj2>)>>()
  tagged_tuple<tag::in1(I1), tag::in2(I2), tag::out(O)>
    transform(I1 first1, S1 last1, I2 first2, O result,
              F binary_op, Proj1 proj1 = Proj1{}, Proj2 proj2 = Proj2{});

// A.2 (deprecated):
-template <InputRange Rng, InputIterator I, WeaklyIncrementable O, class F,
+template <InputRange Rng, InputIterator I, WeaklyIncrementable O, CopyConstructible F,
    class Proj1 = identity, class Proj2 = identity>
  requires Writable<O, indirect_result_of_t<F&(
    projected<iterator_t<Rng>, Proj1>, projected<I, Proj2>>)>()
  tagged_tuple<tag::in1(safe_iterator_t<Rng>), tag::in2(I), tag::out(O)>
  transform(Rng&& rng1, I first2, O result,
            F binary_op, Proj1 proj1 = Proj1{}, Proj2 proj2 = Proj2{});

template <InputIterator I1, Sentinel<I1> S1, InputIterator I2, Sentinel<I2> S2,
-    WeaklyIncrementable O, class F, class Proj1 = identity, class Proj2 = identity>
+    WeaklyIncrementable O, CopyConstructible F, class Proj1 = identity, class Proj2 = identi
  requires Writable<O, indirect_result_of_t<F&(projected<I1, Proj1>,
    projected<I2, Proj2>)>>()
  tagged_tuple<tag::in1(I1), tag::in2(I2), tag::out(O)>
  transform(I1 first1, S1 last1, I2 first2, S2 last2, O result,
            F binary_op, Proj1 proj1 = Proj1{}, Proj2 proj2 = Proj2{});

-template <InputRange Rng1, InputRange Rng2, WeaklyIncrementable O, class F,
+template <InputRange Rng1, InputRange Rng2, WeaklyIncrementable O, CopyConstructible F,
    class Proj1 = identity, class Proj2 = identity>
  requires Writable<O, indirect_result_of_t<F&(
    projected<iterator_t<Rng1>, Proj1>, projected<iterator_t<Rng2>, Proj2>)>>()
  tagged_tuple<tag::in1(safe_iterator_t<Rng1>),
               tag::in2(safe_iterator_t<Rng2>),
               tag::out(O)>
  transform(Rng1&& rng1, Rng2&& rng2, O result,
            F binary_op, Proj1 proj1 = Proj1{}, Proj2 proj2 = Proj2{});
```

with identical changes to the declarations in [alg.transform] and [depr.algo.range-and-a-half]

288: “regular function” != `RegularInvocable`

Per the discussion in ericniebler/range-v3#499. `f` is an “operand” of the expression `invoke(f, args...)`, and both an “input” and (if mutable) an “output” thereof. `RegularInvocable`’s requirement that “The `invoke` function call expression shall be equality-preserving” is therefore not sufficient to ensure that the result of invoking `f` with `args...` depends only on the value of `args...`.

This fundamentally breaks equational reasoning in the algorithms, e.g., I cannot prove that the result of calling `sort(range, comp)` on a sequence is a sorted sequence if the comparator can be stateful. We need to restore the equivalence of regular functions and `RegularInvocable`s.

Proposed Resolution:

Modify [concepts.lib.callable.regularinvocable] to read:

```
 1 The invoke function call expression shall be equality-preserving
+  and shall not modify the function object or the arguments.
   (4.1.1). [ Note: This requirement supersedes the annotation in the
   definition of Invocable. —end note ]
```

299: `value_type` of classes with member `element_type`

`element_type` in `std::pointer_traits` and in the smart pointers is the type of the object the pointer denotes, not the value type of that object (i.e., it can be a cv-qualified type). The specialization of `value_type` that uses `typename T::element_type` should strip cv-qualifiers from the type.

Proposed Resolution:

Modify [iterator.assoc.types.value_type] as follows:

```
 [...]

 2 A Readable type has an associated value type that can be accessed with the value_type_t al
   template.

     [...]
     template <class T>
       requires requires { typename T::value_type; }
     struct value_type<T>
       : enable_if<is_object<typename T::value_type>::value, typename T::value_type> { };

     template <class T>
       requires requires { typename T::element_type; }
     struct value_type<T>
-      : enable_if<is_object<typename T::element_type>::value, typename T::element_type> { };
+      : enable_if<
+          is_object<typename T::element_type>::value,
+          remove_cv_t<typename T::element_type>>
+    { };

     template <class T>
       using value_type_t = typename value_type<T>::type;

 [...]

 5 When instantiated with a type I such that I::value_type is valid and denotes a type, [...]

 6 When instantiated with a type I such that I::element_type is valid and denotes a type,
-  value_type<I>::type names that type,
+  value_type<I>::type names the type remove_cv_t<I::element_type>,
   unless it is not an object type (ISO/IEC 14882:2014 §3.9)
   in which case value_type<I> shall have no nested type type. [ Note: Smart pointers like
   shared_ptr<int> are Readable and have an associated value type. But a smart pointer like
   shared_ptr<void> is not Readable and has no associated value type.—end note ]
```

302: `insert_iterator` and `ostreambuf_iterator` don't properly support `*o++ = t;`

See the discussion at CaseyCarter/cmcstl2#60 (comment). Both iterators internally store state that must be updated on each write. We made a blanket change to have output iterators' `operator++(int)` return a copy instead of a reference after the discussion in #137 (and IIRC discussion with LWG at Kona) we broke `insert_iterator` and `ostreambuf_iterator`. The expression `*o++ = t` results in updating the state of the temporary immediately before throwing it on the floor. Later use of the iterator sees the "old" state and does terrible things.

The fix is to change the postfix increment of at least `insert_iterator` and `ostreambuf_iterator`, if not all of the output iterators specified in the TS, to again return by reference as in Standard C++.

Proposed Resolution

Adopt P0541.

307: Incomplete edit to `InputIterator` to support proxy iterators

The proxy iterators paper (P0022) made essential changes to the `Readable` concept to support proxy iterators but neglected to make a similar necessary edit to `InputIterator`. The `InputIterator` concept is currently requiring that the expression `*ci`, where `ci` is a `const`-qualified `InputIterator I`, is convertible to `const value_type_t<I>&`. This requirement is unsatisfiable for some proxy iterator types.

Consider a zip range with a reference type `pair<unique_ptr<int>&, int&>` and a value type `pair<unique_ptr<int>, int>`. There is no valid conversion from the former type to the latter since it would require copying a `unique_ptr`.

Proposed resolution

Adopt the wording in P0541.

309: Missing "Returns:" clause of `sort`/`stable_sort`/`partial_sort`/`nth_element`

Proposed Resolution

Change **`sort`** ([sort]) as follows:

```
 template <RandomAccessIterator I, Sentinel<I> S, class Comp = less<>,
     class Proj = identity>
   requires Sortable<I, Comp, Proj>()
   I sort(I first, S last, Comp comp = Comp{}, Proj proj = Proj{});

 template <RandomAccessRange Rng, class Comp = less<>, class Proj = identity>
     requires Sortable<iterator_t<Rng>, Comp, Proj>()
   safe_iterator_t<Rng>
   sort(Rng&& rng, Comp comp = Comp{}, Proj proj = Proj{});

 1 Effects: Sorts the elements in the range [first,last).
 2 Complexity: O(N log(N)) (where N == last - first) comparisons.
+3 Returns: last.
```

Change **`stable_sort`** ([stable.sort]) as follows:

```
 template <RandomAccessIterator I, Sentinel<I> S, class Comp = less<>,
     class Proj = identity>
   requires Sortable<I, Comp, Proj>()
   I stable_sort(I first, S last, Comp comp = Comp{}, Proj proj = Proj{});

 template <RandomAccessRange Rng, class Comp = less<>, class Proj = identity>
     requires Sortable<iterator_t<Rng>, Comp, Proj>()
   safe_iterator_t<Rng>
```

```
   stable_sort(Rng&& rng, Comp comp = Comp{}, Proj proj = Proj{});

 1 Effects: Sorts the elements in the range [first,last).
 2 Complexity: It does at most N log2(N) (where N == last - first) comparisons; if enough ext
memory is available, it is N log(N).
 3 Remarks: Stable (ISO/IEC 14882:2014 §17.6.5.7).
+4 Returns: last.
```

Change **`partial_sort`** ([partial.sort]) as follows:

```
 template <RandomAccessIterator I, Sentinel<I> S, class Comp = less<>,
     class Proj = identity>
   requires Sortable<I, Comp, Proj>()
   I partial_sort(I first, I middle, S last, Comp comp = Comp{}, Proj proj = Proj{});
```

```
 template <RandomAccessRange Rng, class Comp = less<>, class Proj = identity>
     requires Sortable<iterator_t<Rng>, Comp, Proj>()
   safe_iterator_t<Rng>
   partial_sort(Rng&& rng, iterator_t<Rng> middle, Comp comp = Comp{}, Proj proj = Proj{});
 1 Effects: Places the first middle - first sorted elements from the range [first,last) into
[first,middle). The rest of the elements in the range [middle,last) are placed in an unspecif
order.
 2 Complexity: It takes approximately (last - first) * log(middle - first) comparisons.
+3 Returns: last.
```

Change Nth element ([alg.nth.element]) as follows:

```
 template <RandomAccessIterator I, Sentinel<I> S, class Comp = less<>,
     class Proj = identity>
   requires Sortable<I, Comp, Proj>()
   I nth_element(I first, I nth, S last, Comp comp = Comp{}, Proj proj = Proj{});

 template <RandomAccessRange Rng, class Comp = less<>, class Proj = identity>
     requires Sortable<iterator_t<Rng>, Comp, Proj>()
   safe_iterator_t<Rng>
   nth_element(Rng&& rng, iterator_t<Rng> nth, Comp comp = Comp{}, Proj proj = Proj{});

 1 After nth_element the element in the position pointed to by nth is the element that would
 position if the whole range were sorted, unless nth == last. Also for every iterator i in th
 first,nth) and every iterator j in the range [nth,last) it holds that: invoke(comp, invoke(p
 *j), invoke(proj, *i)) == false.
 2 Complexity: Linear on average.
+3 Returns: last.
```

311: `Common` and `CommonReference` should use `ConvertibleTo` to test for implicit convertibility

Issue # 1: the expressions `common_type_t<T, U>(t())` and `common_reference_t<T, U>(t())` in the `Common` and `CommonReference` concepts could potentially be interpreted as function-style casts depending on the types involved, which could amount to `reinterpret_cast`s. That was not the intention.

The intention was to test that expressions of type `T` and `U` can be converted to the common type (or the common reference). The fix is to simply test that requirement instead with `ConvertibleTo`.

Issue # 2: One of the potential uses for `common_(type|reference)` is to have a way to declare monomorphic functions that satisfy e.g., `IndirectRelation` for use with the std:: algorithms. Indeed, `IndirectReference` requires the invocable to be callable with common-reference parameters. Consider:

```
auto rng = view::zip(vec1, vec2); // from range-v3
using R = iter_common_reference_t<iterator_t<decltype(rng)>>;
// Note: R might be different than decltype(*begin(rng)) for, e.g. a proxy iterator.
auto pred = [](R a, R b) { return a < b; };
ranges::sort( rng, pred );
```

Right now all the higher order functions are more or less broken because they just do `pred(*here, *there)`. For an iterator whose reference type is explicitly-but-not-implicitly convertible to the common reference type, the call will pass the concept check but fail to instantiate. That's Bad.

As above, the fix is to change `Common` and `CommonReference` to use `ConvertibleTo`. That requires *implicit* convertibility, rather than the explicit convertibility that was being checked with the function-style cast.

Note: The underlying traits `common_type` and `common_reference` only require explicit convertibility. We think it's fine, indeed essential for generic code, for the concepts to be more strict than the underlying type traits.

Proposed Resolution

Change the definition of `CommonReference` in [concepts.lib.corelang.commonref]/1 as follows:

```
 template <class T, class U>
 concept bool CommonReference() {
-  return requires(T (&t)(), U (&u)()) {
-    typename common_reference_t<T, U>;
-    typename common_reference_t<U, T>;
-    requires Same<common_reference_t<T, U>, common_reference_t<U, T>>();
-    common_reference_t<T, U>(t());
-    common_reference_t<T, U>(u());
-  };
+  return
+    Same<common_reference_t<T, U>, common_reference_t<U, T>>() &&
+    ConvertibleTo<T, common_reference_t<T, U>>() &&
+    ConvertibleTo<U, common_reference_t<T, U>>();
 }
```

Note: this removes the `typename common_reference_t...` constraints since they are superfluous.

Change the definition of `Common` in [concepts.lib.corelang.common]/1 as follows:

```
 template <class T, class U>
 concept bool Common() {
-   return CommonReference<const T&, const U&>() &&
-    requires(T (&t)(), U (&u)()) {
-    typename common_type_t<T, U>;
-    typename common_type_t<U, T>;
-    requires Same<common_type_t<U, T>, common_type_t<T, U>>();
-    common_type_t<T, U>(t());
-    common_type_t<T, U>(u());
-    requires CommonReference<add_lvalue_reference_t<common_type_t<T, U>>,
-                             common_reference_t<add_lvalue_reference_t<const T>,
-                                                add_lvalue_reference_t<const U>>>();
- };
+ return
```

```
+   Same<common_type_t<T, U>, common_type_t<U, T>>() &&
+   ConvertibleTo<T, common_type_t<T, U>>() &&
+   ConvertibleTo<U, common_type_t<T, U>>() &&
+   CommonReference<add_lvalue_reference_t<const T>,
+                   add_lvalue_reference_t<const U>>() &&
+   CommonReference<add_lvalue_reference_t<common_type_t<T, U>>,
+                   common_reference_t<add_lvalue_reference_t<const T>,
+                                      add_lvalue_reference_t<const U>>>()
}
```

Note: this removes the `typename common_type_t...` constraints since they are superfluous. It also fixes a bug where the original `Common` definition was trying to form the types `const T&` and `const U&`. That fails for *cv* `void`. We would like `Common<void, void>()` to be `true`.

316: `copy_if` "Returns" clause incorrect

It's currently specified to return `{last, result + (last - first)}`, the second part of which makes no sense since fewer than `last - first` elements may be copied.

Proposed Resolution:

Add a new paragraph before [alg.copy]/8 and change the following paragraphs to read:

```
 template <InputIterator I, Sentinel<I> S, WeaklyIncrementable O, class Proj = identity,
     IndirectPredicate<projected<I, Proj>> Pred>
   requires IndirectlyCopyable<I, O>()
   tagged_pair<tag::in(I), tag::out(O)>
     copy_if(I first, S last, O result, Pred pred, Proj proj = Proj{});

 template <InputRange Rng, WeaklyIncrementable O, class Proj = identity,
     IndirectPredicate<projected<iterator_t<Rng>, Proj>> Pred>
   requires IndirectlyCopyable<iterator_t<Rng>, O>()
   tagged_pair<tag::in(safe_iterator_t<Rng>), tag::out(O)>
     copy_if(Rng&& rng, O result, Pred pred, Proj proj = Proj{});
```

```
+-?- Let N be the number of iterators i in the range [first,last) for which the condition
+     invoke(pred, invoke(proj, *i)) holds.
-  8 Requires: The ranges [first,last) and [result,result + (last - first)) shall not overlap
+  8 Requires: The ranges [first,last) and [result,result + N) shall not overlap.
   9 Effects: Copies all of the elements referred to by the iterator i in the range [first,la
     for which invoke(pred, invoke(proj, *i)) is true.
- 10 Returns: {last, result + (last - first)}.
+ 10 Returns: {last, result + N}.
  11 Complexity: Exactly last - first applications of the corresponding predicate and project
  12 Remarks: Stable (ISO/IEC 14882:2014 §17.6.5.7).
```

317: `is_nothrow_indirectly_movable` could be `true` even when `iter_move` is `noexcept(false)`

`is_nothrow_indirectly_movable` is currently defined as follows:

```
template <class In, class Out>
    requires IndirectlyMovable<In, Out>()
struct is_nothrow_indirectly_movable<In, Out> :
```

```
  std::integral_constant<bool,
    is_nothrow_constructible<value_type_t<In>, rvalue_reference_t<In>>::value &&
    is_nothrow_assignable<value_type_t<In> &, rvalue_reference_t<In>>::value &&
    is_nothrow_assignable<reference_t<Out>, rvalue_reference_t<In>>::value &&
    is_nothrow_assignable<reference_t<Out>, value_type_t<In>>::value>
{ };
```

This doesn't take into account that `iter_move(declval<In&>())` could be `noexcept(false)`. That makes it misleading at best. (Never mind the fact that `iter_move` should not ever be `noexcept(false)`).

`is_nothrow_indirectly_movable` should be defined in terms of `iter_move`.

Proposed resolution

Change the definition of `is_nothrow_indirectly_movable` in [iterator.traits]/9 as follows:

```
 template <class In, class Out>
     requires IndirectlyMovable<In, Out>()
 struct is_nothrow_indirectly_movable<In, Out> :
   std::integral_constant<bool,
+    noexcept(ranges::iter_move(declval<In&>())) &&
     is_nothrow_constructible<value_type_t<In>, rvalue_reference_t<In>>::value &&
     is_nothrow_assignable<value_type_t<In> &, rvalue_reference_t<In>>::value &&
     is_nothrow_assignable<reference_t<Out>, rvalue_reference_t<In>>::value &&
     is_nothrow_assignable<reference_t<Out>, value_type_t<In>>::value>
 { };
```

318: `common_iterator::operator->` does not specify its return type

Its return type is listed as "*see below*", and its *Effects:* clause describes the return values, but we should be more explicit about what the actual return type is in all cases.

Proposed Resolution

After [common.iter.op.star]/2 add a subsection title "**`common_iterator::operator->`**" with stable name [common.iter.op.ref]. Then change the following specification of `common_iterator::operator->` as follows.

```
-see below operator->() const requires Readable<I>();
+decltype(auto) operator->() const requires Readable<I>();

 1 Requires: !is_sentinel

-2 Effects: Given an object i of type I
+2 Effects: Equivalent to:

-(2.1) — if I is a pointer type or if the expression i.operator->() is well-formed,
-        this function returns iter.
+(2.1) — return iter; if I is a pointer type or if the expression iter.operator->() is
+        well-formed.

-(2.2) — Otherwise, if the expression *iter is a glvalue, this function is equivalent
-        to return addressof(*iter);
+(2.2) — Otherwise, if the expression *iter is a glvalue:

+       auto&& tmp = *iter;
+       return addressof(tmp);
```

```
-(2.3) — Otherwise, this function returns a proxy object of an unspecified type
-        equivalent to the following:
+(2.3) — Otherwise, return proxy(*iter); where proxy is the exposition-only class:

     class proxy {
      value_type_t<I> keep_;
      proxy(reference_t<I>&& x)
        : keep_(std::move(x)) {}
     public:
       const value_type_t<I>* operator->() const {
          return addressof(keep_);
       }
    };
```

321: Concepts that use type traits are inadvertently subsuming them

Implementors may want the freedom to implement the concepts directly in terms of the same compiler intrinsics that are used to implement the traits. However, by specifying the concepts directly in terms of the type traits, we create a subsumption relationship between the concept and the trait. Implementations no longer have the freedom to *not* use the library trait because that has observable effects on overload resolution.

We propose eliminating the subsumption relationships, thereby giving implementors the freedom to innovate under the as-if rule.

Proposed Resolution

Change [concepts.lib.corelang.same] as follows:

```
 template <class T, class U>
 concept bool Same() {
-  return see below ;
+  return is_same<T, U>::value; // see below
 }

-1 Same<T, U>() is satisfied if and only if T and U denote the same
-  type.
+1 There need not be any subsumption relationship between Same<T, U>() and
+  is_same<T, U>::value.

 2 Remarks: For the purposes of constraint checking, Same<T, U>()
   implies Same<U, T>().
```

Change [concepts.lib.corelang.derived] as suggested in #255.

Change [concepts.lib.corelang.convertibleto] as suggested in #167.

Change [concepts.lib.corelang.integral] as follows:

```
 template <class T>
 concept bool Integral() {
-  return is_integral<T>::value;
+  return is_integral<T>::value; // see below
 }

+1 There need not be any subsumption relationship between Integral<T>() and
+  is_integral<T>::value.
```

Change [concepts.lib.corelang.signedintegral] as follows:

```
 template <class T>
 concept bool SignedIntegral() {
-  return Integral<T>() && is_signed<T>::value;
+  return Integral<T>() && is_signed<T>::value; // see below
 }

+1 There need not be any subsumption relationship between SignedIntegral<T>()
+  and is_signed<T>::value.
```

330: Argument deduction constraints are specified incorrectly

See #155 for detailed discussion, but basically we've been operating under the faulty assumption that the following:

```
requires(const T& t) {  {t} -> Same<const T&>; }
```

was trivially satisfied. It is not. Argument deduction constraints are handled as if the placeholder were made the argument type of an invented function `f`, like `void f(Same<const T&>)`, and then a call were made like `f(t)`. That causes the type of `t` to decay.

This should instead be written as:

```
requires(const T& t) {  {t} -> Same<const T&>&&; }
```

That prevents argument decay from happening, which recovers the value category information.

We need to review and amend *all* the definitions and uses of concepts in the ranges ts.

(In our defense, this slipped through because early concepts implementations didn't do argument deduction constraints, so we invented a workaround, and the workaround was buggy because we didn't understand the model.)

Proposed Resolution:

Update section "Concept `Assignable`" ([concepts.lib.corelang.assignable]) and "Concept `Destructible`" ([concepts.lib.object.destructible]) as specified in P0547R1.

Update sections "Concept `Boolean`" [concepts.lib.compare.boolean]), "Concept `EqualityComparable`" ([concepts.lib.compare.equalitycomparable]), "Concept `StrictTotallyOrdered`" ([concepts.lib.compare.stricttotallyordered]) as specified in #155.

Update section "Concept `Readable`" ([iterators.readable]) as follows (also fixes #339):

```
template <class I>
concept bool Readable() {
- return Movable<I>() && DefaultConstructible<I>() &&
-   requires(const I& i) {
+ return requires {
    typename value_type_t<I>;
    typename reference_t<I>;
```

```
    typename rvalue_reference_t<I>;
-   { *i } -> Same<reference_t<I>>;
-   { ranges::iter_move(i) } -> Same<rvalue_reference_t<I>>;
  } &&
  CommonReference<reference_t<I>, value_type_t<I>&>() &&
  CommonReference<reference_t<I>, rvalue_reference_t<I>>() &&
  CommonReference<rvalue_reference_t<I>, const value_type_t<I>&>();
}
```

Change section “Concept `WeaklyIncrementable`” ([iterators.weaklyincrementable]) as follows:

```
template <class I>
concept bool WeaklyIncrementable() {
  return Semiregular<I>() &&
  requires(I i) {
    typename difference_type_t<I>;
    requires SignedIntegral<difference_type_t<I>>();
-   { ++i } -> Same<I&>; // not required to be equality preserving
+   { ++i } -> Same<I>&; // not required to be equality preserving
    i++; // not required to be equality preserving
  };
}
```

Change section “Concept `Incrementable`” ([iterators.incrementable]) as follows:

```
template <class I>
concept bool Incrementable() {
  return Regular<I>() &&
    WeaklyIncrementable<I>() &&
    requires(I i) {
-     { i++ } -> Same<I>; // not required to be equality preserving
+     { i++ } -> Same<I>&&; // not required to be equality preserving
    };
}
```

Change section “Concept `SizedSentinel`” ([iterators.sizedsentinel]) as follows:

```
template <class S, class I>
concept bool SizedSentinel() {
  return Sentinel<S, I>() &&
    !disable_sized_sentinel<remove_cv_t<S>, remove_cv_t<I>> &&
    requires(const I& i, const S& s) {
-     { s - i } -> Same<difference_type_t<I>>;
-     { i - s } -> Same<difference_type_t<I>>;
+     { s - i } -> Same<difference_type_t<I>>&&;
+     { i - s } -> Same<difference_type_t<I>>&&;
    };
}
```

Take the definition of concept `InputIterator` ([iterators.input]) from P0541.

Change section “Concept `BidirectionalIterator`” ([iterators.bidirectional]) as follows:

```
template <class I>
concept bool BidirectionalIterator() {
  return ForwardIterator<I>() &&
```

```
    DerivedFrom<iterator_category_t<I>, bidirectional_iterator_tag>() &&
    requires(I i) {
-     { --i } -> Same<I&>;
-     { i-- } -> Same<I>;
+     { --i } -> Same<I>&;
+     { i-- } -> Same<I>&&;
    };
}
```

Change section “Concept `RandomAccessIterator`” ([iterators.random.access]) as follows:

```
template <class I>
concept bool RandomAccessIterator() {
  return BidirectionalIterator<I>() &&
    DerivedFrom<iterator_category_t<I>, random_access_iterator_tag>() &&
    StrictTotallyOrdered<I>() &&
    SizedSentinel<I, I>() &&
    requires(I i, const I j, const difference_type_t<I> n) {
-     { i += n } -> Same<I&>;
-     { j + n } -> Same<I>;
-     { n + j } -> Same<I>;
-     { i -= n } -> Same<I&>;
-     { j - n } -> Same<I>;
-     { j[n] } -> Same<reference_t<I>>;
+     { i += n } -> Same<I>&;
+     { j + n } -> Same<I>&&;
+     { n + j } -> Same<I>&&;
+     { i -= n } -> Same<I>&;
+     { j - n } -> Same<I>&&;
+     j[n];
+     requires Same<decltype(j[n]), reference_t<I>>();
    };
}
```

Change section “Concept `UniformRandomNumberGenerator`” ([rand.req.urng]) as follows:

```
template <class G>
concept bool UniformRandomNumberGenerator() {
- return requires(G g) {
-   { g() } -> UnsignedIntegral; // not required to be equality preserving
-   { G::min() } -> Same<result_of_t<G&()>>;
-   { G::max() } -> Same<result_of_t<G&()>>;
+ return Invocable<G&>() &&
+   UnsignedIntegral<result_of_t<G&()>>() &&
+   requires {
+     { G::min() } -> Same<result_of_t<G&()>>&&;
+     { G::max() } -> Same<result_of_t<G&()>>&&;
    };
}
```

331: Reorder requirements in concept `Iterator`

When passing a `std::vector<MoveOnly>` to `sort`, overload resolution causes the evaluation of `Iterator<std::vector<MoveOnly>>`, which tests the vector type for assignability. Vectors of move-only types are *not* assignable, but their assignment operators are not properly constrained. That leads to a hard error rather than a concept check failure.

The problem can be mostly avoided by testing for dereferenceability *first* in concept `Iterator`, as shows in the Proposed Resolution below.

Proposed Resolution

Change concept `Iterator` ([iterators.iterator]) as follows:

```
 template <class I>
 concept bool Iterator() {
-   return WeaklyIncrementable<I>() &&
-     requires(I i) {
-       { *i } -> auto&&; // Requires: i is dereferenceable
-     };
+   return requires(I i) {
+     { *i } -> auto&&; // Requires: i is dereferenceable
+   } && WeaklyIncrementable<I>();
 }
```

345: US 2 (006): 2.1.1: Update ranged-for-loop wording

This is US national body comment 2 on the PDTS.

Editorial Comment

The wording no longer matches the more thoroughly reviewed form in the C++17 Working Draft.

Proposed Change

Entirely replace this wording with the proposed wording for C++17.

Proposed Resolution

Accept the wording changes in P0662R0 “Wording for Ranges TS Issue 234 / US-2.”

354: JP 1 (015): 6.9.2.2/1: Range doesn’t require begin

This is JP national body comment 1 on the PDTS.

Editorial Comment

`ranges::begin(t)` is missing in “requires” for `Range`.

Proposed Change

Add `ranges::begin(t);` to “requires”.

Proposed Resolution

Modify the definition of the `Range` concept in [ranges.range] as follows:

```
 template <class T>
 concept bool Range() {
   return requires(T&& t) {
+    ranges::begin(t);
     ranges::end(t);
   };
 }
```

357: JP 3 (018): 7.4.4/5: transform does not include projection calls in Complexity

This is JP national body comment 3 on the PDTS.

Editorial Comment

Lack of consideration about projection in Complexity.

Proposed Change

Add a description about projection.

Proposed Resolution

Modify [alg.transform]/5 as follows:

```
-Complexity: Exactly N applications of op or binary_op.
+Complexity: Exactly N applications of the projection(s) and of op or binary_op.
```